22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 100

int main() {

    printf("Valgrind Memory Management Demo\n");
    printf("-------------------------------\n");

    /* Allocate memory dynamically */
    char *message = malloc(SIZE);

    if (message == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    strcpy(message, "Hello from dynamically allocated memory!");

    printf("Message: %s\n", message);

    /* Allocate another block */
    int *numbers = malloc(5 * sizeof(int));

    if (numbers == NULL) {
        printf("Memory allocation failed.\n");
        free(message);
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        numbers[i] = (i + 1) * 10;
    }

    printf("Numbers: ");
```

```c
    printf("Numbers: ");

    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*
     * INTENTIONAL MEMORY LEAK:
     *
     * message and numbers are not freed.
     *
     * Valgrind will detect these allocations
     * as memory that was not released.
     */

    printf("Program completed.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:          :~$ nano Skill22a.c
sanjana3027@DESKTOP-QJF5AGU:          :~$ gcc Skill22a.c -o S22a
sanjana3027@DESKTOP-QJF5AGU:          :~$ ./S22a
Valgrind Memory Management Demo
------------------------------
Message: Hello from dynamically allocated memory!
Numbers: 10 20 30 40 50
Program completed.
sanjana3027@DESKTOP-QJF5AGU:          :~$
```

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Code:

```c
#include <stdio.h>
#include <string.h>

int main() {

    char command[100];

    printf("Enter command: ");

    if (fgets(command, sizeof(command), stdin) == NULL) {
        printf("INPUT_ERROR\n");
        return 1;
    }

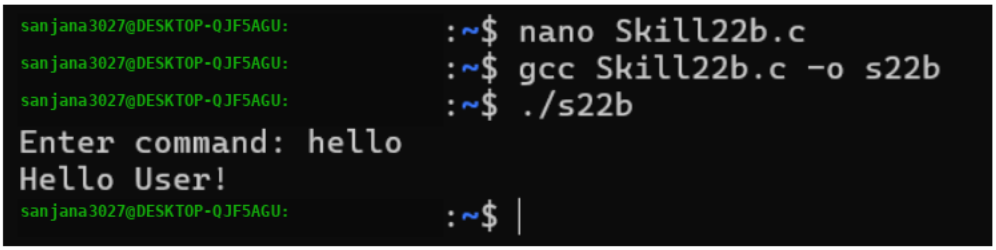
    command[strcspn(command, "\n")] = '\0';

    if (strlen(command) == 0) {
        printf("EMPTY_COMMAND\n");
        return 1;
    }

    if (strcmp(command, "hello") == 0) {
        printf("Hello User!\n");
    }
    else if (strcmp(command, "status") == 0) {
        printf("System is running.\n");
    }
    else if (strcmp(command, "exit") == 0) {
        printf("Exit command received.\n");
    }
    else {
        printf("INVALID_COMMAND\n");
        return 1;
    }

    return 0;
}
```

Result:


sanjana3027@DESKTOP-QJF5AGU: :~$ nano Skill22b.c
sanjana3027@DESKTOP-QJF5AGU: :~$ gcc Skill22b.c -o s22b
sanjana3027@DESKTOP-QJF5AGU: :~$ ./s22b
Enter command: hello
Hello User!
sanjana3027@DESKTOP-QJF5AGU: :~$